

WEB APP ASCII STRUCTURED DOCUMENT

Created for the web application by Kristupas Gelzinis and Olek Suchodolski

Detailed LaTeX source, coding-compiler blueprint, asset inventory, and PDF-ready documentation

extbfGenerated date/time: Saturday, 2026-05-23 09:00:55 EEST (UTC+03:00) extbfLocation: Vilnius, Lithuania (LTU) extbfCoordinates used: 54.6872 N, 25.2797 E extbfWorkspace rule: all referenced files use relative paths only. extbfPrivacy note: personal identity numbers appearing in the uploaded archive are intentionally not reproduced.

Attached Visual Assets



1 Master ASCII Map

```
ROOT: WORKBEE / SAPTERA WEB APP
|
+-- AUTHORS
|   +-- Kristupas Gelzinis
|   +-- Olek Suchodolski
|
+-- SOURCE ARTIFACTS
|   +-- site-structure-for-ACII-format.txt           [CSS/Tailwind compiled bundle]
|   +-- 598576741_..._n.jpg                         [visual reference]
|   +-- kristupas.jpg                               [author portrait asset]
|   +-- olek.jpeg                                    [author portrait asset]
|   +-- Save all chat ... (1).pdf                   [prior prompt/output archive]
|
+-- APPLICATION SHELL
|   +-- Dark design system
|   +-- Utility-first CSS runtime
|   +-- Sidebar navigation
|   +-- Admin panels and cards
|   +-- Forms, inputs, selects, buttons
|   +-- Charts through Recharts-compatible selectors
|   +-- Command palette through cmdk selectors
|   +-- Toast notifications through Radix-style data attributes
|
+-- CODING COMPILATOR
|   +-- Parser: reads bundle and class grammar
|   +-- Mapper: converts CSS tokens into UI components
|   +-- Generator: emits React/TypeScript component scaffolds
|   +-- Validator: checks state attributes and accessibility hooks
|   +-- Builder: compiles app, assets, and PDF documentation
```

2 Application Component Tree

```
APP
|
+--- <RootLayout>
|   +--- <ThemeProvider mode="dark">
|   +--- <Metadata date="Vilnius EEST">
|   +--- <AssetRegistry uploads="relative paths only">
|
+--- <PublicSurface>
|   +--- <HeroBanner>
|   +--- <FeatureCards>
|   +--- <CallToAction>
|
+--- <AdminPanel class="admin-panel">
|   +--- <AdminSidebar class="admin-sidebar">
|       |   +--- <NavGroup>
|       |   +--- <NavItem data-active>
|       |   +--- <CollapseRail>
|       +--- <AdminContent>
|           +--- <AdminCard class="admin-card">
|               +--- <DataTable>
|               +--- <ChartGrid recharts-*>
|               +--- <FormStack>
|
+--- <InteractionLayer>
    +--- <CommandMenu cmdk-*>
    +--- <Dialog data-state="open|closed">
    +--- <ToastViewport data-swipe="move|cancel|end">
    +--- <Tooltip data-side="top|right|bottom|left">
```

3 Source Artifact Measurements

Artifact	Size / Shape	Role
uploads/site-structure-for-ACII-format.txt	99,708 characters	Compiled CSS/Tailwind source for reverse engineering
uploads/Save all chat as it is with all prompts and gener... (1).pdf	468,612 bytes	Prior prompt/output archive
uploads/598576741_10232762808913842_12509486344704256781_JPEG	1430 x 1078 JPEG	Visual reference asset
uploads/kristupas.jpg	1080 x 1086 JPEG	Kristupas portrait asset
uploads/olek.jpeg	800 x 800 JPEG	Olek portrait asset

4 Detected CSS and UI Signals

- admin: 67 occurrences

sidebar: 73 occurrences

card: 17 occurrences

toast: 4 occurrences

recharts: 28 occurrences

cmdk: 30 occurrences

button: 38 occurrences

input: 47 occurrences
- select: 43 occurrences

tooltip: 12 occurrences

accordion: 8 occurrences

dialog: 1 occurrences

navigation: 4 occurrences

roll-d20: 3 occurrences

5 1. Identity and ownership layer

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

[01] IDENTITY AND OWNERSHIP LAYER

```
|
+-- Purpose: define the identity and ownership layer as a compilable unit.
+-- Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+-- Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+-- Acceptance:
|   +-- Uses relative paths only.
|   +-- Preserves dark UI contrast.
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
|   +-- Parse tokens.
|   +-- Map component selectors.
|   +-- Generate scaffold.
|   +-- Compile app.
|   +-- Export documentation.
```

5.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

6 2. Input artifact inventory

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

[02] INPUT ARTIFACT INVENTORY

```
|
+-- Purpose: define the input artifact inventory as a compilable unit.
+-- Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+-- Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+-- Acceptance:
|   +-- Uses relative paths only.
|   +-- Preserves dark UI contrast.
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
|   +-- Parse tokens.
|   +-- Map component selectors.
|   +-- Generate scaffold.
|   +-- Compile app.
|   +-- Export documentation.
```

6.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.

- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

7 3. Design token compiler

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

[03] DESIGN TOKEN COMPILER

```
|
+-- Purpose: define the design token compiler as a compilable unit.
+-- Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+-- Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+-- Acceptance:
|   +-- Uses relative paths only.
|   +-- Preserves dark UI contrast.
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
    +-- Parse tokens.
    +-- Map component selectors.
    +-- Generate scaffold.
    +-- Compile app.
    +-- Export documentation.
```

7.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

8 4. Component grammar

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

[04] COMPONENT GRAMMAR

```
|
+-- Purpose: define the component grammar as a compilable unit.
+-- Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+-- Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+-- Acceptance:
|   +-- Uses relative paths only.
|   +-- Preserves dark UI contrast.
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
    +-- Parse tokens.
    +-- Map component selectors.
    +-- Generate scaffold.
    +-- Compile app.
```

```
+++ Export documentation.
```

8.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

9 5. Administrative console

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

```
[05] ADMINISTRATIVE CONSOLE
```

```
|
+++ Purpose: define the administrative console as a compilable unit.
+++ Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+++ Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+++ Acceptance:
|   +++ Uses relative paths only.
|   +++ Preserves dark UI contrast.
|   +++ Supports admin, public, and interaction surfaces.
|   +++ Emits readable errors for missing tokens or assets.
+++ Developer Tasks:
|   +++ Parse tokens.
|   +++ Map component selectors.
|   +++ Generate scaffold.
|   +++ Compile app.
|   +++ Export documentation.
```

9.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

10 6. Navigation and sidebar system

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

```
[06] NAVIGATION AND SIDEBAR SYSTEM
```

```
|
+++ Purpose: define the navigation and sidebar system as a compilable unit.
+++ Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+++ Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+++ Acceptance:
|   +++ Uses relative paths only.
|   +++ Preserves dark UI contrast.
```

```
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
    +-- Parse tokens.
    +-- Map component selectors.
    +-- Generate scaffold.
    +-- Compile app.
    +-- Export documentation.
```

10.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

11 7. Command palette and search

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

[07] COMMAND PALETTE AND SEARCH

```
|
+-- Purpose: define the command palette and search as a compilable unit.
+-- Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+-- Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+-- Acceptance:
|   +-- Uses relative paths only.
|   +-- Preserves dark UI contrast.
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
    +-- Parse tokens.
    +-- Map component selectors.
    +-- Generate scaffold.
    +-- Compile app.
    +-- Export documentation.
```

11.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

12 8. Forms and validation

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

[08] FORMS AND VALIDATION

```

|
+-- Purpose: define the forms and validation as a compilable unit.
+-- Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+-- Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+-- Acceptance:
|   +-- Uses relative paths only.
|   +-- Preserves dark UI contrast.
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
    +-- Parse tokens.
    +-- Map component selectors.
    +-- Generate scaffold.
    +-- Compile app.
    +-- Export documentation.

```

12.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

13 9. Charts and analytics

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

```

[09] CHARTS AND ANALYTICS
|
+-- Purpose: define the charts and analytics as a compilable unit.
+-- Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+-- Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+-- Acceptance:
|   +-- Uses relative paths only.
|   +-- Preserves dark UI contrast.
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
    +-- Parse tokens.
    +-- Map component selectors.
    +-- Generate scaffold.
    +-- Compile app.
    +-- Export documentation.

```

13.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

14 10. Notifications and toast flow

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

[10] NOTIFICATIONS AND TOAST FLOW

```
|
+-- Purpose: define the notifications and toast flow as a compilable unit.
+-- Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+-- Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+-- Acceptance:
|   +-- Uses relative paths only.
|   +-- Preserves dark UI contrast.
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
    +-- Parse tokens.
    +-- Map component selectors.
    +-- Generate scaffold.
    +-- Compile app.
    +-- Export documentation.
```

14.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

15 11. Animation engine

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

[11] ANIMATION ENGINE

```
|
+-- Purpose: define the animation engine as a compilable unit.
+-- Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+-- Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+-- Acceptance:
|   +-- Uses relative paths only.
|   +-- Preserves dark UI contrast.
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
    +-- Parse tokens.
    +-- Map component selectors.
    +-- Generate scaffold.
    +-- Compile app.
    +-- Export documentation.
```

15.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.

- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

16 12. Responsive layout rules

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

[12] RESPONSIVE LAYOUT RULES

```
|
+-- Purpose: define the responsive layout rules as a compilable unit.
+-- Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+-- Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+-- Acceptance:
|   +-- Uses relative paths only.
|   +-- Preserves dark UI contrast.
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
    +-- Parse tokens.
    +-- Map component selectors.
    +-- Generate scaffold.
    +-- Compile app.
    +-- Export documentation.
```

16.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

17 13. Security and privacy notes

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

[13] SECURITY AND PRIVACY NOTES

```
|
+-- Purpose: define the security and privacy notes as a compilable unit.
+-- Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+-- Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+-- Acceptance:
|   +-- Uses relative paths only.
|   +-- Preserves dark UI contrast.
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
    +-- Parse tokens.
    +-- Map component selectors.
    +-- Generate scaffold.
    +-- Compile app.
```

```
+++ Export documentation.
```

17.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

18 14. Build pipeline for coding compiler

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

```
[14] BUILD PIPELINE FOR CODING COMPILATOR
```

```
|
+++ Purpose: define the build pipeline for coding compiler as a compilable unit.
+++ Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+++ Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+++ Acceptance:
|   +++ Uses relative paths only.
|   +++ Preserves dark UI contrast.
|   +++ Supports admin, public, and interaction surfaces.
|   +++ Emits readable errors for missing tokens or assets.
+++ Developer Tasks:
|   +++ Parse tokens.
|   +++ Map component selectors.
|   +++ Generate scaffold.
|   +++ Compile app.
|   +++ Export documentation.
```

18.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

19 15. Implementation backlog

This section translates the uploaded bundle into implementation-ready ASCII architecture for the coding compiler. The intent is to preserve the visible design language while turning it into a repeatable build plan.

```
[15] IMPLEMENTATION BACKLOG
```

```
|
+++ Purpose: define the implementation backlog as a compilable unit.
+++ Inputs: CSS variables, utility classes, data-state selectors, and uploaded assets.
+++ Outputs: typed components, validated styles, route-level documentation, and PDF evidence.
+++ Acceptance:
|   +++ Uses relative paths only.
|   +++ Preserves dark UI contrast.
```

```
|   +-- Supports admin, public, and interaction surfaces.
|   +-- Emits readable errors for missing tokens or assets.
+-- Developer Tasks:
    +-- Parse tokens.
    +-- Map component selectors.
    +-- Generate scaffold.
    +-- Compile app.
    +-- Export documentation.
```

19.1 Implementation Notes

- Treat the CSS bundle as the canonical UI vocabulary and avoid inventing unrelated design tokens.
- Represent each visible module as a small component with clear props and explicit state attributes.
- Keep the admin surface separate from the public surface so authentication and audit logic remain isolated.
- Use the uploaded portraits and reference image as optional assets, not hard dependencies for runtime logic.
- Validate all generated output with a build step and produce a PDF documentation artifact after successful compilation.

20 Class Inventory Sample

-bottom-12	bg-amber-400	bg-popover
-left-12	bg-amber-500	bg-primary
-ml-2	bg-amber-500\{}/15	bg-primary\{}/10
-ml-4	bg-background	bg-primary\{}/20
-mt-1	bg-background\{}/80	bg-primary\{}/5
-mt-4	bg-background\{}/95	bg-purple-500\{}/15
-mx-1	bg-black	bg-red-500\{}/10
-my-2	bg-black\{}/10	bg-red-700
-right-12	bg-black\{}/50	bg-rose-500\{}/10
-right-3	bg-black\{}/60	bg-rose-500\{}/5
-top-12	bg-black\{}/80	bg-secondary
-top-3	bg-blue-500\{}/10	bg-sidebar
-translate-x-1\{}/2	bg-blue-500\{}/15	bg-sidebar-border
-translate-x-px	bg-blue-500\{}/20	bg-transparent
-translate-y-1\{}/2	bg-border	bg-white
-z-10	bg-card	bg-yellow-500\{}/10
absolute	bg-card\{}/50	bg-yellow-500\{}/20
active\{}	bg-destructive	block
admin-card	bg-destructive\{}/10	border
admin-input	bg-destructive\{}/20	border-0
admin-panel	bg-destructive\{}/5	border-2
after\{}	bg-foreground	border-\{}
align-middle	bg-foreground\{}/10	border-amber-500\{}/30
animate-\{}	bg-gradient-to-r	border-b
animate-bounce	bg-gray-500\{}/20	border-blue-500\{}/20
animate-in	bg-green-500	border-blue-500\{}/30
animate-pulse	bg-green-500\{}/10	border-border
animate-roll-d20	bg-green-600	border-border\{}/50
animate-spin	bg-input	border-collapse
aria-disabled\{}	bg-muted	border-dashed
aria-selected\{}	bg-muted\{}/20	border-destructive
aspect-square	bg-muted\{}/30	border-destructive\{}/20
aspect-video	bg-muted\{}/40	border-destructive\{}/50
backdrop-blur	bg-muted\{}/50	border-green-500\{}/20
backdrop-filter	bg-muted\{}/60	border-green-500\{}/40
basis-full	bg-orange-500\{}/10	border-input
bg-\{}	bg-pink-500\{}/10	border-1
bg-accent	bg-pink-500\{}/15	border-1-2

border-l-primary	divide-\{}	font-mono
border-l-transparent	divide-border	font-normal
border-none	divide-y	font-sans
border-pink-500\{}/30	drop-shadow-xl	font-semibold
border-primary	duration-100	from-amber-500
border-primary\{}/20	duration-1000	from-primary\{}/10
border-primary\{}/30	duration-200	gap-0\{}.5
border-primary\{}/40	duration-300	gap-1
border-primary\{}/50	duration-500	gap-1\{}.5
border-purple-500\{}/30	ease-in	gap-2
border-r	ease-in-out	gap-2\{}.5
border-red-500\{}/20	ease-linear	gap-3
border-rose-500	ease-out	gap-4
border-rose-500\{}/20	fade-in-0	gap-5
border-sidebar-border	file\{}	gap-6
border-t	fill-current	gap-7
border-t-transparent	fill-muted	gap-8
border-transparent	fill-primary	googleapis.com/css2?family=Outfit
border-white	fill-rose-500	grid
border-y	filter	grid-cols-1
border-yellow-500\{}/20	first\{}	grid-cols-2
border-yellow-500\{}/30	fixed	grid-cols-3
bottom-0	flex	group
bottom-4	flex-1	group-\{}
bottom-6	flex-\{}	group-data-\{}
break-words	flex-col	group-focus-within\{}/menu-item\{}
capitalize	flex-col-reverse	group-has-\{}
caption-bottom	flex-row	group-hover\{}
cursor-default	flex-shrink-0	group-hover\{}/menu-item\{}
cursor-pointer	flex-wrap	group.destructive
cursor-text	focus-visible\{}	group\{}/day
dark	focus\{}	group\{}/field
dark\{}	font-black	group\{}/field-group
data-\{}	font-bold	group\{}/input-group
destructive\{\}\{}	font-extrabold	
disabled\{}	font-medium	

21 Minimal Compiler Pseudocode

```
function compileWebApp(sourceBundle, assets):
  tokens = parseCssVariables(sourceBundle)
  classes = parseUtilityClasses(sourceBundle)
  components = inferComponents(classes, tokens)
  routes = generateRoutes(["/", "/admin", "/admin/analytics", "/admin/settings"])
  ui = generateReactComponents(components, assets)
  validateAccessibility(ui)
  validateRelativePaths(assets)
  buildApplication(ui, routes, tokens)
  exportPdfDocumentation("main.pdf")
  return { status: "compiled", pdf: "main.pdf" }
```

22 Deliverables Checklist

- `main.tex`: this long ASCII-structured LaTeX document.
- `main.pdf`: compiled PDF deliverable generated from this source.
- `uploads/site-structure-for-ACII-format.txt`: raw app structure source.
- `uploads/*.jpg` and `uploads/*.jpeg`: visual reference and portrait assets.
- Coding-compiler blueprint: parser, mapper, generator, validator, builder, and PDF exporter.